Zero-shot Generative Linguistic Steganography

Ke Lin[1] **Yiyang Luo**[2] **Zijian Zhang**[1] **Ping Luo**[1]
[1]Tsinghua University
[2]Nanyang Technological University
{leonard.keilin, zijianzhang510}@gmail.com
lawrenceluoyy@outlook.com
luop@tsinghua.edu.cn

Abstract

Generative linguistic steganography attempts to hide secret messages into covertext. Previous studies have generally focused on the statistical differences between the covertext and stegotext, however, ill-formed stegotext can readily be identified by humans. In this paper, we propose a novel zero-shot approach based on in-context learning for linguistic steganography to achieve better perceptual and statistical imperceptibility. We also design several new metrics and reproducible language evaluations to measure the imperceptibility of the stegotext. Our experimental results indicate that our method produces 1.926× more innocent and intelligible stegotext than any other method[1].

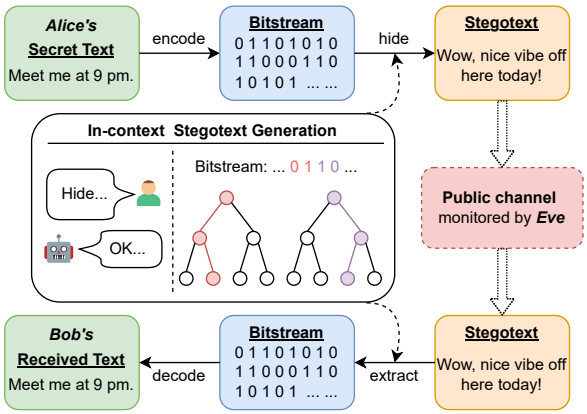

Figure 1: Generative linguistic steganography pipeline.

1 Introduction

Data transmission is generally secured using encryption algorithms to create an unrecognizable ciphertext for secure data transmission (Bellare et al., 1997). Nonetheless, messages transmitted as ciphertexts may easily arouse suspicion from attackers, and eavesdropping may place a threat on the messages being sent. Once the communication channel is discovered, the attackers could alter the messages in any way, causing significant damage.

Steganography is the key to ensuring communication privacy by concealing sensitive messages within monitored channels. General steganography involves embedding a secret message into a public multimedia carrier, such as text (Krishnan et al., 2017), image (Subramanian et al., 2021; Tao et al., 2019; Guo et al., 2015), audio (Lin et al., 2018), and video (Liu et al., 2019) to create a stego-carrier that is then transmitted via the public channel. As shown in Fig. 1, the problem of secure and covert communication can be viewed as a message transmission between two parties, Alice and Bob, along with a malicious party, Eve. Eve will monitor the entire transmission and prevent unintelligible messages from being transmitted as Alice attempts to send secret information to Bob. The solution to this problem is to use a steganographic function to embed the secret message into a stego-carrier controlled by the shared secret key. The stego-carrier is then sent to Bob without alarming Eve, and Bob can extract the secret message from the stego-carrier by using the corresponding extraction function. A key objective in this scenario is to ensure that covert communications are imperceptible.

In public channels, text is commonly used, however, text contains relatively little redundant information (Zipf, 1999) when compared to other media, so embedding large amounts of information can be challenging. Moreover, steganalysis tools can detect stegotext containing secret messages based on machine learning or statistical characteristics (Taleby Ahvanooey et al., 2019). To address this problem, modern steganography utilizes machine learning techniques to enhance its imperceptibility.

Over the last few decades, researchers have devised many techniques to realize generation-based steganography (Xiang et al., 2022), including rule-based (Chapman and Davida, 1997), statistical (Moraldo, 2014; Shniperov and Nikitina, 2016),

[1]The project is available in https://github.com/leonardodalinky/zero-shot-GLS.

Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 5168–5182
June 16-21, 2024 ©2024 Association for Computational Linguistics

and combinations of these methods (Luo et al., 2016). Recent developments (Otter et al., 2021) in machine learning and natural language processing have led to improved performance in linguistic steganography. It involves the use of neural networks to learn the statistical distribution of a large number of sentences, and then generate steganographic sentences from the learned language model (Dai and Cai, 2019; Yang et al., 2019b).

Prior works have typically formulated statistical imperceptibility as the distance between the covertext distribution and stegotext distribution, however, there is no clear and simple method of measuring perceptual imperceptibility. Furthermore, many studies (Ziegler et al., 2019; Yang et al., 2021) have demonstrated that metrics for assessing statistical imperceptibility fail to measure perceptual imperceptibility and produce inaccurate or even incorrect results. The discrepancy between the statistical and perceptual imperceptibility prevents the practical application of linguistic steganography.

Our Contributions. In this work, we propose a novel zero-shot linguistic steganography method with both improved perceptual and statistical imperceptibility. Our new method is built based on the in-context learning of large language models (LLM), which utilize some samples of the covertext to generate more intelligible stegotext using the question-answer paradigm (QA). Initially, the secret text is encoded in binary bitstreams, which are then used to generate the stegotext. At each timestep t, the LM computes the distribution of the t-th token and embeds the bitstream into the generated text by Huffman encoding. To demonstrate the efficacy of our proposed method compared with different supervised and training-free approaches, we develop several new metrics and conduct reproducible experiments of language evaluation.

In summary, our contributions are as follows:

- We present a zero-shot approach for linguistic steganography based on in-context learning using samples of the covertext.
- We improve both the binary coding process and the embedding process by introducing several novel techniques.
- We design several metrics and language evaluations to evaluate both the perceptual and statistical imperceptibility, whereas our method produces more innocent and intelligible stegotext compared to all the previous methods.

2 Background

2.1 Generative Linguistic Steganography

General text generation task aims at creating token sequence $\boldsymbol{x} = [x_1, \ldots, x_n]$ from the joint probability of language model $\mathbf{P}_{\text{LM}}(\boldsymbol{x})$. The generation process can often be viewed as an auto-regression:

$$\begin{aligned}\mathbf{P}_{\text{LM}}(\boldsymbol{x}) &= \prod_{t=1}^{n} \mathbf{P}_{\text{LM}}(x_t \mid x_1, \ldots, x_{t-1}) \\ &= \prod_{t=1}^{n} \mathbf{P}_{\text{LM}}(x_t \mid x_{<t})\end{aligned} \tag{1}$$

The goal of linguistic steganography is to send a secret message $\boldsymbol{m} \in \{0,1\}^l \subset \mathcal{M}$ to the receiver through a monitored public channel. In generative linguistic steganography, the sender and receiver share an embedding algorithm $f_{\text{emb}} : \mathcal{P}_{\text{LM}} \times \mathcal{M} \mapsto \mathcal{P}_{\text{stega}}$, which transforms $\mathbf{P}_{\text{LM}}$ into the steganographic distribution $\mathbf{P}_{\text{stega}}$ controlled by message $\boldsymbol{m}$. It is then used to achieve covert communication by generating *stegotext* $\boldsymbol{y} \sim \mathbf{P}_{\text{stega}}$. Similarly, there exists an extraction process $f_{ext} : \mathcal{P}_{\text{stega}} \mapsto \mathcal{M}$ that extracts the message $\boldsymbol{m}$ from the stegotext $\boldsymbol{y}$.

2.2 Statistical Imperceptibility

To measure the statistical imperceptibility under surveillance by an eavesdropper, Cachin (1998) proposed to use the Kullback-Leibler divergence (KLD) between the distributions of natural text $\mathbf{P}_{\text{true}}$ and stegotext $\mathbf{P}_{\text{stega}}$. Since we approximate the distribution of true text $\mathbf{P}_{\text{true}}$ using a language model $\mathbf{P}_{\text{LM}}$, there is a gap between the distribution predicted by the model and the true distribution of text. Hence, we need to approximate this gap by finding an upper bound. We can formulate the upper bound of imperceptibility by using Jensen–Shannon divergence (JSD) instead:

$$\begin{aligned}\text{JSD}(\mathbf{P}_{\text{true}} \parallel \mathbf{P}_{\text{stega}})^{\frac{1}{2}} \leq \text{JSD}(\mathbf{P}_{\text{true}} \parallel \mathbf{P}_{\text{LM}})^{\frac{1}{2}} \\ + \text{JSD}(\mathbf{P}_{\text{LM}} \parallel \mathbf{P}_{\text{stega}})^{\frac{1}{2}}\end{aligned} \tag{2}$$

Assuming that the language model is successful in modeling the covertext distribution, we can simplify the statistical imperceptibility as $\text{JSD}(\mathbf{P}_{\text{LM}} \parallel \mathbf{P}_{\text{stega}})$. This strong assumption can be considered true due to language models' performance. In other words, it is feasible to measure imperceptibility using only a language model $\mathbf{P}_{\text{LM}}$ and a steganographic model $\mathbf{P}_{\text{stega}}$.

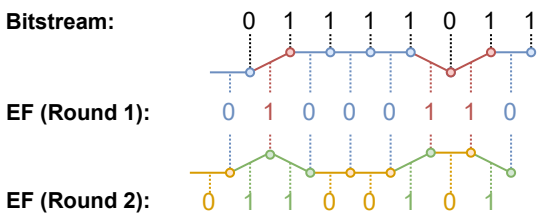

Figure 2: Example of EF coding. The red and green lines represent that the coding in bitstream changed (from 0 to 1, or from 1 to 0) and therefore will be assigned as 1 for the next iteration. The blue and yellow line represents that the coding in bitstream did not change (from 0 to 0, or from 1 to 1) and therefore will be assigned as 0 for the next iteration. Bitstream is processed through several rounds until the fewest 1s exist.

3 Methodology

In this section, we present the overall framework of our approach, consisting of three modules: (a) *Codec* module to (de)compress secret text into (from) bitstreams, (b) *Embedding* module to select candidate words from the distribution of stegotext, (c) *In-Context Stegotext Generation* module to approximate the joint distribution of covertext. The pipeline of our framework is illustrated in Fig. 1. Figure 3 also explains the details of the embedding module and the in-context stegotext generation.

3.1 Codec

The sender in linguistic steganography wishes to send secret text to the receiver, which must first be converted into a binary bitstream. We refer to this process as encoding and decoding (codec).

Variable-length Coding. There are two categories of encoding algorithms: fixed-length coding (FLC) and variable-length coding (VLC). We employ Huffman coding of the VLC family for token-level encoding, as it is a relatively simple algorithm and has a sub-optimal compression ratio. Tokens are represented in binary form using the Huffman code table conditioned on $\mathbf{P}_{\text{LM}}(x_t \mid x_{<t})$.

Edge Flipping Coding. We observe that more frequent occurrences of sequential zeros in the bitstreams can improve the quality of the generated text in the embedding module due to the local ordering of the Huffman tree (i.e., the left branch always has a higher probability than the right branch). Hence, we introduce Edge Flipping coding (EF), inspired by the differential encoding of communication (Weber, 1978), on top of the Huffman process to achieve better text generation performance.

In EF coding, the bitstream is transformed to record the position at which the bits change based on the edge-triggered flip-flops in electronics. We apply multi-round EF coding (e.g. 16 rounds) and the bitstream with the least number of 1s is retained. A simple example of EF is provided in Fig. 2. The analysis of EF coding is presented in Section 4.6.

3.2 Embedding

The goal of the embedding module is to hide and extract bitstreams within regular sentences.

Hide & Extract. During each phase of the generative operation, the LM yields a set of *candidate words*. These words are determined based on the subsequent token's probability and can potentially serve as alternatives for the next possible word to be generated. Then the LM will determine the length of the binary bitstream that can be embedded based on the number of candidate words.

Algorithm 1 shows the brief algorithm of the hiding process. The binary representation of selected words in the conditional word distribution is obtained using the Huffman encoding algorithm. It should be noted that this Huffman process is independent of the aforementioned Huffman coding in the Codec section. Following Dai and Cai (2019), we use the threshold τ to reduce perplexity by pruning words with lower probabilities.

Algorithm 1 Information Hiding Algorithm

Input: Bitstream $\boldsymbol{m}$, threshold τ.
Output: stegotext $\boldsymbol{y} = [y_1, \ldots, y_n]$.

Timestep $t \leftarrow 1$, output sentence $\boldsymbol{y} \leftarrow \emptyset$
while not the end of $\boldsymbol{m}$ **do**
  ▷ `Compute conditional probs`
  $\boldsymbol{p} \leftarrow \mathbf{P}_{\text{stega}}(x_t \mid x_{<t})$
  ▷ `Prune candidate words`
  $\boldsymbol{c} \leftarrow [c_i \mid \boldsymbol{p}(c_i) \geq \tau]$
  ▷ `Huffman encoding`
  $H \leftarrow \text{Huffman}(\boldsymbol{c}, \boldsymbol{p})$
  ▷ `Select candidate`
  $y_t \leftarrow$ Word $c \in H$ whose binary representation matches the prefix of $\boldsymbol{m}$
  $\boldsymbol{y} \leftarrow \boldsymbol{y} \cup \{y_t\}, t \leftarrow t + 1$
end while

As the reverse process of hiding, the information extraction process extracts steganographic information from stegotext. With a few minor differences,